The Coding Interview Patterns Playbook

Taras Goriachko

Section 0 — Welcome & Course Orientation

Requirements

- Basic knowledge of any programming language - ideally Kotlin
- Optional: Understanding of the Gradle build system*
- **Preinstalled on computer:**
  - IntelliJ IDEA 2024.2.4 (Community Edition)**
  - Java 17***

---

*http://udemy.com/course/mastering-gradle/

*https://www.jetbrains.com/idea/download/?section=mac

**https://adoptium.net/en-GB/temurin/releases/?version=17&os=mac&arch=aarch64&package=jre

Welcome to the Course

- Why "Patterns" instead of "Problems"?
- Moving from memorization to intuition
- The 80/20 Rule: 15 patterns solve 80% of interviews

"Don't practice until you get it right. Practice until you can't get it wrong."

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition
- The 80/20 Rule: 15 patterns solve 80% of interviews

"Don't practice until you get it right. Practice until you can't get it wrong."

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition 💡
- The 80/20 Rule: 15 patterns solve 80% of interviews

"Don't practice until you get it right. Practice until you can't get it wrong."

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition 💡
- The 80/20 Rule: 15 patterns solve 80% of interviews 📈

"Don't practice until you get it right. Practice until you can't get it wrong."

How to Use This Course Effectively

- **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text
  2. **Trace:** Dry-run the logic on a small input
  3. **Implement:** Code the solution without looking at hints
- **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

How to Use This Course Effectively

- **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔎
  2. **Trace:** Dry-run the logic on a small input
  3. **Implement:** Code the solution without looking at hints
- **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

How to Use This Course Effectively

- **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔎
  2. **Trace:** Dry-run the logic on a small input ✏️
  3. **Implement:** Code the solution without looking at hints
- **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

How to Use This Course Effectively

- **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text 🔎
  2. **Trace:** Dry-run the logic on a small input ✏️
  3. **Implement:** Code the solution without looking at hints ⌨️
- **If Stuck:** Start with a brute-force solution, optimize what you can, and carefully listen to your interviewer.

Interview Mindset: Thinking in Patterns

- Stop looking at the "Story" (e.g., "Alice has 3 apples...")
- Start looking at the **Data Flow**

Problem Trait	**Pattern Strategy**
Finding a sub-segment in a list	Sliding Window
Searching a sorted range	Binary Search
Shortest path in unweighted graph	BFS

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear
- **Mistake 2:** Forgetting to state Big O complexity
- **Mistake 3:** Not testing for $n = 0$ or $n = 1$

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** Forgetting to state Big O complexity
- **Mistake 3:** Not testing for $n = 0$ or $n = 1$

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** Forgetting to state Big O complexity ❌
- **Mistake 3:** Not testing for $n = 0$ or $n = 1$

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** Forgetting to state Big O complexity ❌
- **Mistake 3:** Not testing for $n = 0$ or $n = 1$ ❌

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Course Structure

We will discuss

Section 1 — Two Pointers: The Simplest Powerful Pattern

What is the Two Pointers Pattern?

- **The Core Idea:** Use two indices (pointers) to iterate through a data structure simultaneously.
- **Why it works:** It reduces $O(n^2)$ nested loops to a single $O(n)$ pass.
- **Common Triggers:**
  - The array is **Sorted**.
  - You need to find a pair, a triplet, or a subarray.
  - You need to "filter" or "rearrange" an array in-place.

Pattern Mechanics: Opposite Ends

When searching for a target in a **Sorted** array:

1. **Left Pointer:** Starts at index 0.
2. **Right Pointer:** Starts at index $n - 1$.
3. **Decision:**
   - If $sum > target$: Move Right pointer left ($\leftarrow$) to decrease sum.
   - If $sum < target$: Move Left pointer right ($\rightarrow$) to increase sum.

Hero Problem: Two Sum (Sorted)

```
fun twoSum(numbers: IntArray, target: Int): IntArray {
    var left = 0
    var right = numbers.size - 1

    while (left < right) {
        val sum = numbers[left] + numbers[right]
        when {
            sum == target -> return intArrayOf(left + 1, right + 1)
            sum < target -> left++
            else -> right--
        }
    }
    return intArrayOf()
}
```

Time Complexity: $O(n)$
Space Complexity: $O(1)$

Two Pointers – Key Takeaways

- ✅ **Space Efficiency:** Usually allows for $O(1)$ extra space.
- ✅ **In-place operations:** Perfect for "Move Zeroes" or "Remove Duplicates."
- ✅ **Pointer Variants:**
  - **Opposite Ends:** (Left, Right) $\rightarrow$ Searching/Sorting.
  - **Same Direction:** (Slow, Fast) $\rightarrow$ Removing elements.

Problem 1: Two-Pointer Approach

Problem: Find indices of two numbers in a **sorted** array that sum to a target.

The Algorithm

1. Initialize $i = 0, j = n - 1$
2. Calculate $sum = nums[i] + nums[j]$
3. If $sum == target$, return indices.
4. If $sum < target$, move $i \rightarrow$
5. If $sum > target$, move $j \leftarrow$

Kotlin Implementation

```
fun twoSum(nums: IntArray, target: Int):
      IntArray {
    var i = 0
    var j = nums.size - 1

    while (i < j) {
        val sum = nums[i] + nums[j]
        when {
            sum == target -> return
                  intArrayOf(i, j)
            sum < target -> i++
            else -> j--
        }
    }
    return intArrayOf()
}
```

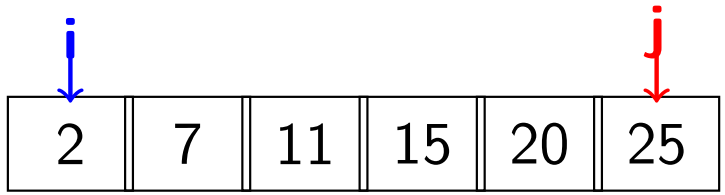
Complexity

- **Time:** $O(N)$
- **Space:** $O(1)$

Two Sum: Visual Trace

Target = 35

1.

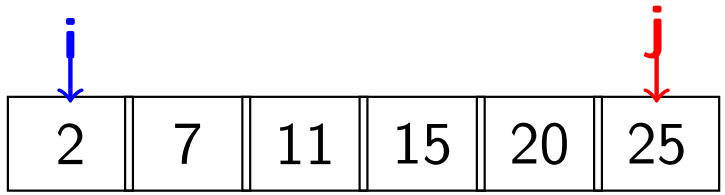


$2 + 25 = 27$ (Too Low, $i++$)

Two Sum: Visual Trace

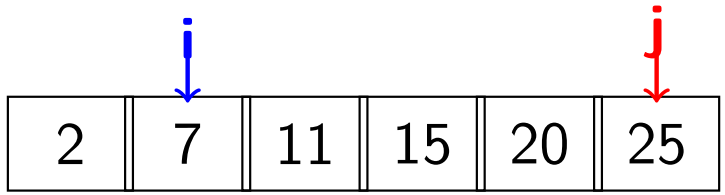
Target = 35

1.



$2 + 25 = 27$ (Too Low, $i++$)

2.

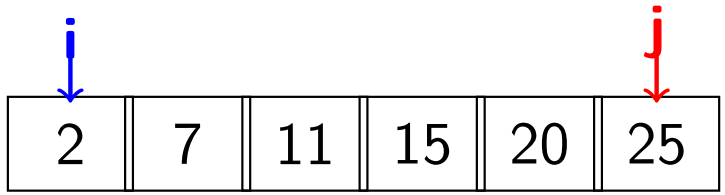


$7 + 25 = 32$ (Too Low, $i++$)

Two Sum: Visual Trace

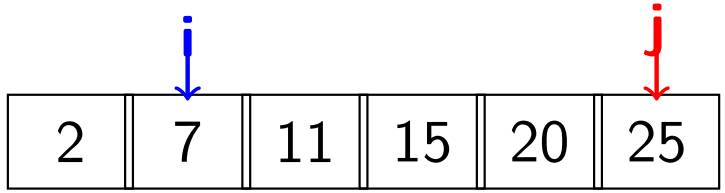
Target = 35

1.



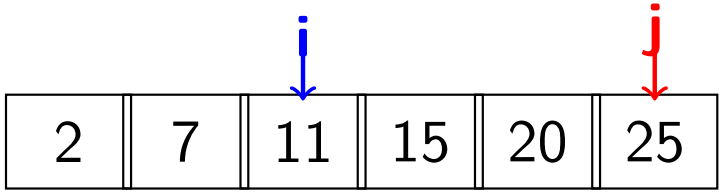
$2 + 25 = 27$ (Too Low, $i++$)

2.



$7 + 25 = 32$ (Too Low, $i++$)

3.

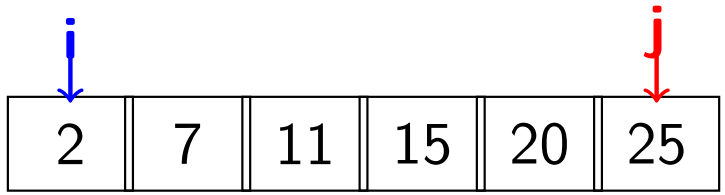


$11 + 25 = 36$ (Too High, $j--$)

Two Sum: Visual Trace

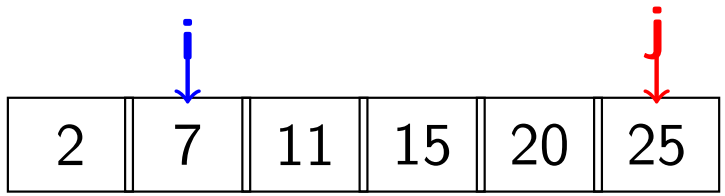
Target = 35

1.



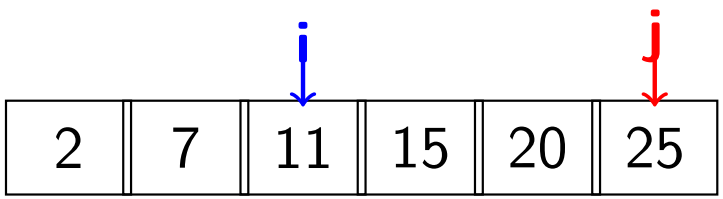
$2 + 25 = 27$ (Too Low, $i++$)

2.



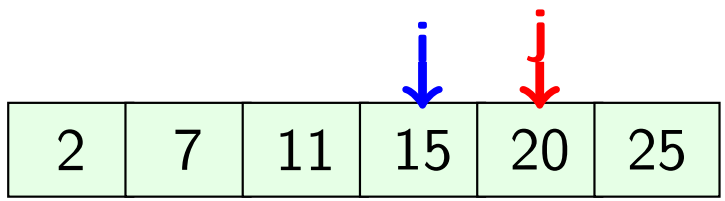
$7 + 25 = 32$ (Too Low, $i++$)

3.



$11 + 25 = 36$ (Too High, $j--$)

4.



15 + 20 = 35!

Problem 2: Container With Most Water

Goal: Find two lines that together with the x-axis forms a container, such that the container contains the most water.

The Algorithm

1. $i = 0, j = n - 1$.
2. $Area = \min(h[i], h[j]) \times (j - i)$.
3. Update $maxArea$ if current is larger.
4. **Crucial:** Move the pointer pointing to the **shorter** line.

Kotlin Implementation

```kotlin
fun maxArea(height: IntArray): Int {
    var max = 0
    var i = 0
    var j = height.size - 1

    while (i < j) {
        val h = minOf(height[i], height[j])
        val w = j - i
        max = maxOf(max, h * w)

        if (height[i] < height[j]) i++
        else j--
    }
    return max
}
```

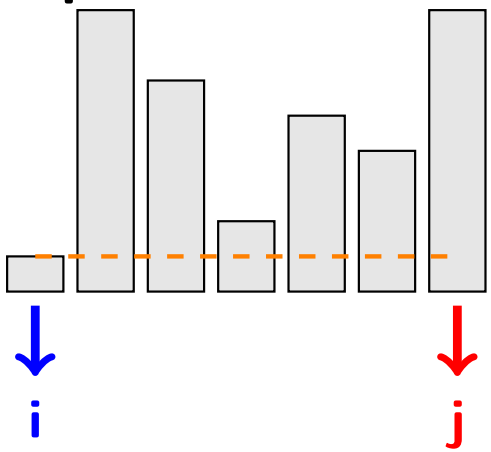
Complexity

- **Time:** $O(N)$
- **Space:** $O(1)$

Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8] **Max Area: 40**

Step 1: Initial State



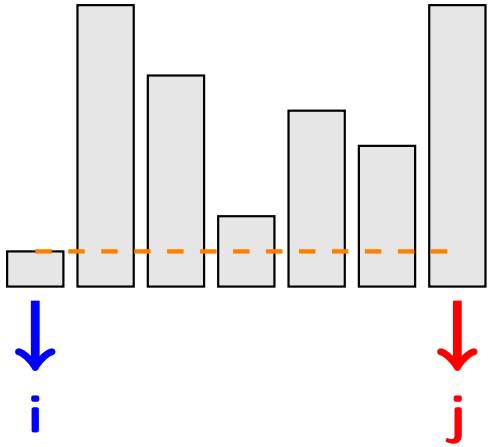
$W = 6, H = 1 \rightarrow$ **Area** $= 6$

$h[i] < h[j] \implies i++$

Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8] **Max Area: 40**

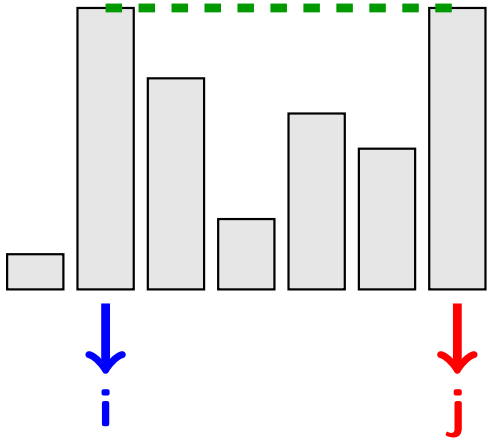
Step 1: Initial State



$W = 6, H = 1 \rightarrow$ **Area = 6**

$h[i] < h[j] \implies i++$

Step 2: Found Max

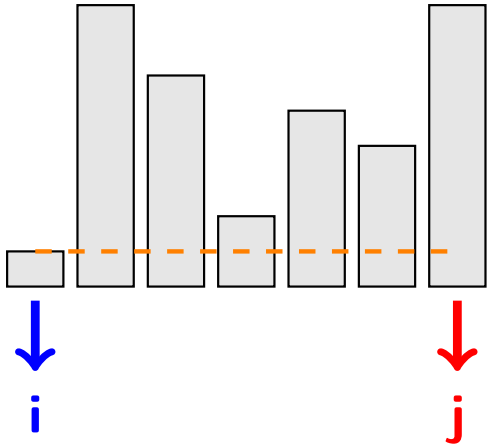


$W = 5, H = 8 \rightarrow$ **Area = 40**

Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8] **Max Area: 40**

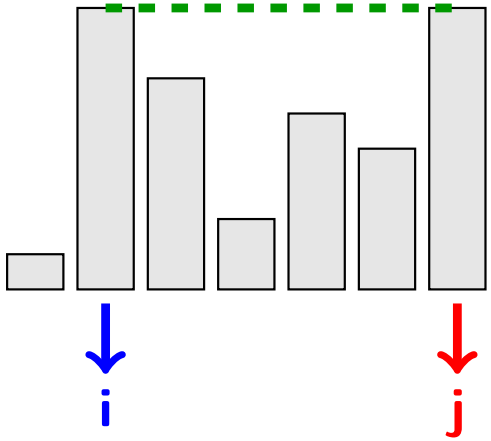
Step 1: Initial State



$W = 6, H = 1 \rightarrow$ **Area = 6**

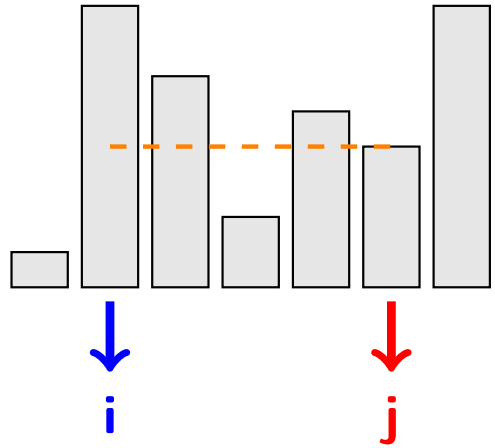
$h[i] < h[j] \implies i++$

Step 2: Found Max



$W = 5, H = 8 \rightarrow$ **Area = 40**

Step 3: Trying for more...



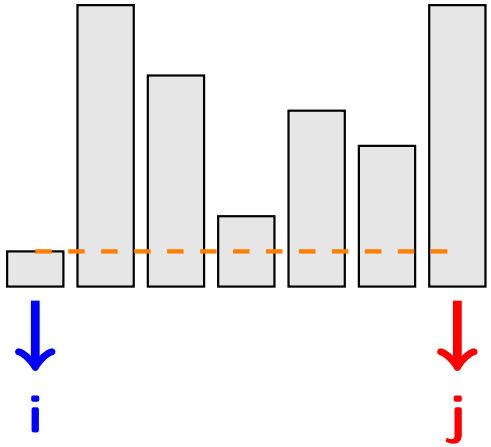
$W = 4, H = 4 \rightarrow$ **Area = 16**

Max remains 40

Container With Most Water: Trace

Heights: [1, 8, 6, 2, 5, 4, 8] Max Area: 40

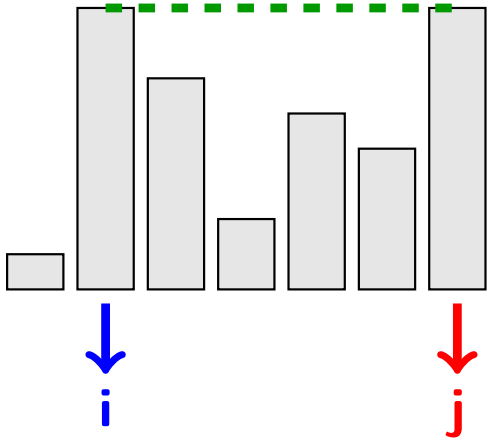
Step 1: Initial State



$W = 6, H = 1 \rightarrow$ **Area = 6**

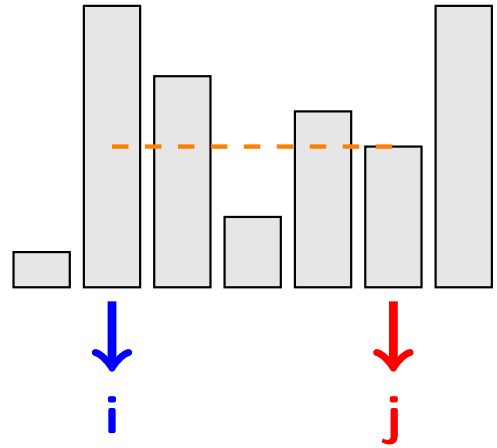
$h[i] < h[j] \implies i++$

Step 2: Found Max



$W = 5, H = 8 \rightarrow$ **Area = 40**

Step 3: Trying for more...



$W = 4, H = 4 \rightarrow$ **Area = 16**

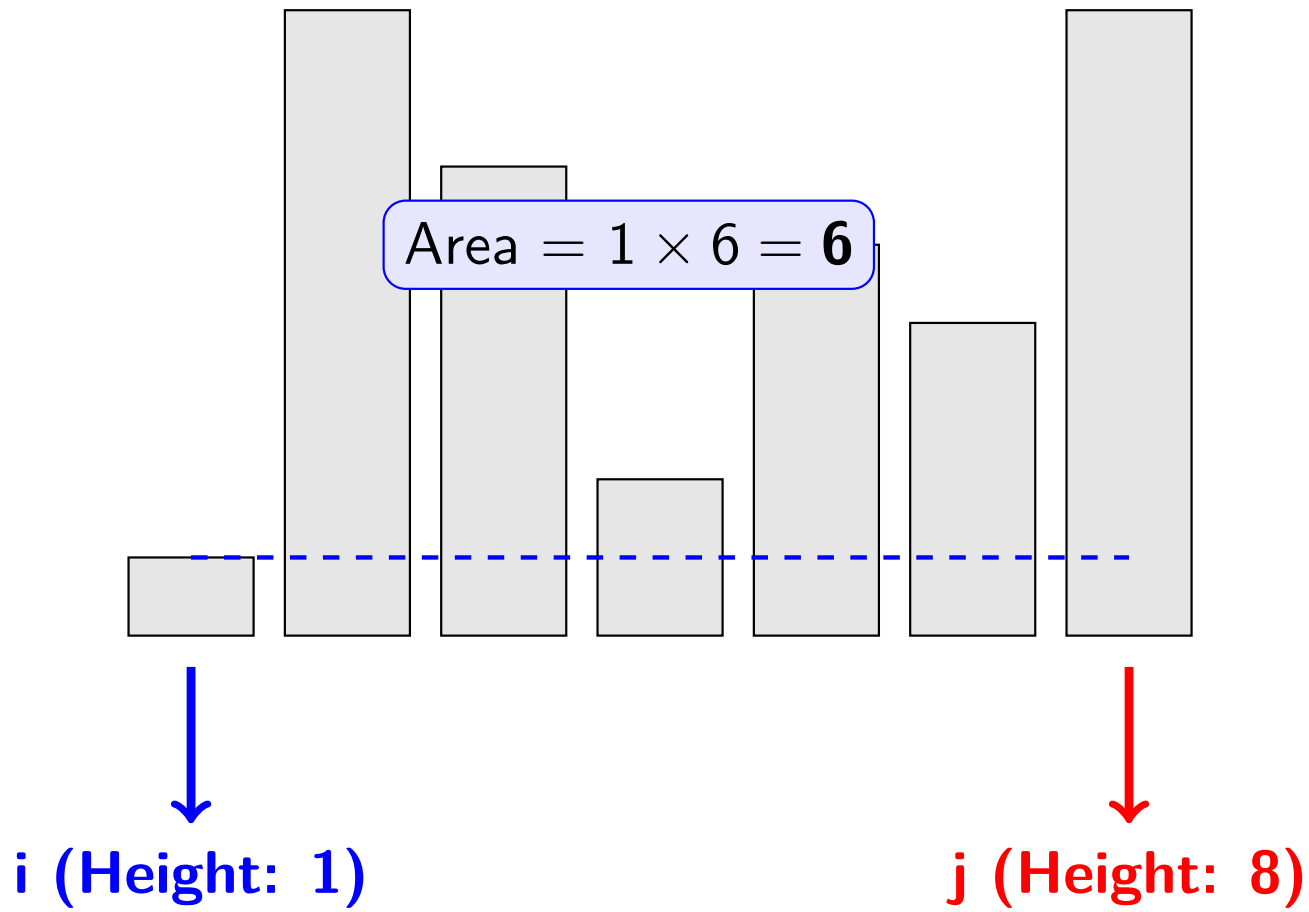
Max remains 40

Intuition

Moving the taller pointer can only decrease area (width shrinks and height is capped by the shorter side). We **must** move the shorter pointer to find a taller container.

Container With Most Water: Iteration 1

Heights: [1, 8, 6, 2, 5, 4, 8]



Logic

$h[i] < h[j]$ $(1 < 8)$. To find a larger area, we move the **shorter**